

Моё решение я разделил на несколько основных этапов: сначала я посмотрел на данные и увидел, что platform записан в разных форматах (iPhone отдельно от ios, desktop отдельно от web), поэтому сначала свёл всё к трём значениям: web, android, ios. Заодно убрал точные дубликаты строк в событиях.

Дальше нашёл проблему с временными диапазонами: часть событий в train происходит уже после конца окна наблюдения куки, а рядом почти всегда встречается captcha\_shown, то есть это уже помеченные случаи. Если такие события не выкинуть, признаки будут смотреть в будущее и офлайн оценка окажется завышенной (то есть прямого вида лик данных), поэтому оставил только события строго внутри окна. Заодно восстановил пропуски в категории, локации и типе продавца по item\_id, у одного объявления они всегда одинаковые!

После этого я перешёл к подробному изучению данных по восьми направлениям, которые выделил из подсказок внизу и своих идей: паузы между событиями, user-agent и платформа, разнообразие того, что кука смотрит, поиск, курсор, возраст куки, пересечения кук по общим объявлениям, сдвиг между train и test. И с этой информации подтвердил в полне реальные вещи, что у ботов короче паузы, курсор (там, где он вообще есть, это только треть кук на web) у ботов собран в плотное пятно вместо равномерного разброса у людей, боты смотрят больше локаций и категорий даже при одинаковой активности, а куки, которые в один день смотрели одно объявление с другими куками, чаще оказываются ботами. Отдельно проверил, что train и test похожи по агрегатам, что помогло сделать вывод, что валидации можно доверять.

На основе этих находок собрал 60 признаков по восьми группам, а после сделал отбор фичей, а именно проверил по очереди, что теряется при удалении каждой группы признаков, на пяти окнах валидации по временн. Паузы, курсор, объём, разнообразие и воронка оказались нужны, а помогает в мобильном сегменте, поиск и признаки куки нейтральны. Также из созданных фичей, пробовал ещё сократить их по важности, но выигрыша это не дало, поэтому оставил все 60 (в реальном проекте надо было бы гораздо подробнее изучить этот вопрос, просто к сожалению не хватило времени(( ))

А после этого разделил куки на три сегмента по доступным данным (web с курсором, web без курсора, мобильные) и попробовал свою идею, которая казалась мне максимально логичной, обучить отдельную модель под каждый сегмент, но лучше не стало, поэтому вернулся к одной модели на всех. После

этого я перебрал побольше моделей абсолютно разных видов: линейные, RandomForest, ExtraTrees, kNN, SVM, MLP и четыре бустинга. Бустинги оказались стабильно впереди и примерно равны друг другу, значит дело было в признаках, а не в выборе модели. Взвешивание классов не помогло, так как метрика ранговая. В итоге построил ансамбль из трёх бустингов (CatBoost, XGBoost, LightGBM), усредняя их ранги, это дало небольшой, но стабильный прирост над любой моделью по отдельности. В реальности я понимаю, что ансамбль только из бустингов, звучит не очень, так как суть бустинга в разных моделях, но так как в моём случае это даёт улучшение, а если ещё покопаться в теории, то можно понять, что на самом деле все три бустинга работают по разному, особенно в построение листьев, поэтому в качестве контекста, почему нет))

Отдельно попробовал ещё две идеи: модель на уровне отдельных событий и моделью устройства, но они обе не дали прироста..